# Practical No: 01

## Aim:

- Write a program to solve the 2-water jug problem by manually providing moves from the user.
- Write a program to implement Missionaries and Cannibals problem.

## Theory:

Questions:

1. why is BFS preferred over DFS here?

Ans: BFS is preferred over DFS because BFS travels level by level and ensures optimal solution, when cost is not present.

2. What happens if boat capacity is increased to 3?

Ans: Increasing the capacity to 3 will increase the number of combinations of travel

and will also give solution in lesser amount of time.

3. Can the water jug problem be solved using BFS instead of manual input?

Ans: Yes, we can solve the water jug problem using BFS but for that we will need a matrix representing all possible states and then use BFS to find the shortest path to reach the goal.

4. How do constraints prune invalid states?

Ans: Constraints prune invalid states in such a way it does not allow the algo. to

go in such a direction which is impossible to be found.

## Implementation:

Water Jug Problem:

Functions:

```
def fillx(a,x):
    if(a[0]<x):
        a[0]=x
    else:
        print("Already filled")
    return a
```

```
def filly(a,y):
    if(a[1]<y):
        a[1]=y
    else:
        print("Already filled")
    return a

def Emptyx(a):
    if(a[0]>0):
        a[0]=0
    else:
        print("Already empty")
    return a

def Emptyy(a):
    if(a[1]>0):
        a[1]=0
    else:
        print("Already empty")
    return a

def pourxy(a,y):
    rem=y-a[1]
    if(a[1]+a[0]<=y):
        a[1]+=a[0]
        a[0]=0
    else:
        a[1]=y
        a[0]-=rem
```

```
    return a


def pouryx(a,x):
    rem=x-a[0]
    if(a[1]+a[0]<=x):
        a[0]+=a[1]
        a[1]=0
    else:
        a[0]=x
        a[1]-=rem
    return a
Main:
x, y = map(int, input("Enter Cap of 2 jugs: ").split())
g=int(input("Enter goal to reach"))
a=[0,0]
case=[]
if(g<=x):
    case.append([g,0])
if(g<=y):
    case.append([0,g])
print("Options you can choose")
print("1. FillX")
print("2. FillY")
print("3. EmptyX")
print("4. EmptyY")
print("5. PourXY")
print("6. PourYX")


while(a not in case):
```

```
    b=int(input("Enter Choice:"))
    match b:
        case 1:
            print("FillX running")
            a=fillx(a,x)
        case 2:
            print("FillY running")
            a=filly(a,y)
        case 3:
            print("EmptyX running")
            a=Emptyx(a)
        case 4:
            print("EmptyY running")
            a=Emptyy(a)
        case 5:
            print("PourXY running")
            a=pourxy(a,y)
        case 6:
            print("PourYX running")
            a=pouryx(a,x)
        case _:
            print("Invalid choice")
            break
    print("Current State",a)



print("Goal Reached")
```

Missionaries and Cannibals problem:

```
from collections import deque
def is_valid(m,c):
    return (m==0 or m>=c) and (((3-m)>=0) or ((3-m)>=(3-c)))


def mc():
    start=(3,3,0)
    goal=(0,0,1)
    d=deque()
    d.append((start,[]))
    visited=set()
    while d:
        (m,c,b),path=d.popleft()
        if((m,c,b)==goal):
            print("Answer Found")
            for p in path:
                print(p)
            return

        if((m,c,b)in visited):
            continue

        visited.add((m,c,b))
        state=()
        for mb,cb in ((1,0),(0,1),(2,0),(0,2),(1,1)): // mb,cb denotes missionary and cannible on
boat
            if(b==0):
                state=(m-mb,c-cb,1)
            else:
```

```
            state=(m+mb,c+cb,0)

        if(0<=state[0]<=3 and 0<=state[1]<=3):
            if(is_valid(state[0],state[1])):
                d.append((state,path+[state]))

mc()
```

## Output:

Options you can choose

1. FillX

2. FillY

3. EmptyX

4. EmptyY

5. PourXY

6. PourYX

FillX running

Current State [3, 0]

PourXY running

Current State [0, 3]

FillX running

Current State [3, 3]

PourXY running

Current State [1, 5]

FillY running

Already filled

Current State [1, 5]

EmptyY running

Current State [1, 0]

Goal Reached

Missionaries and Cannibal problem:

Answer Found

(3, 1, 1)

(3, 2, 0)

(3, 0, 1)

(3, 1, 0)

(1, 1, 1)

(2, 1, 0)

(0, 1, 1)

(1, 1, 0)

(0, 0, 1)

## Result Analysis:

The Output for water jug problem shows which number on entering performs what function and shows the reached state of the solution.

The Output of Missionaries and Cannibal show the number of missionaries, cannibals on the left and show where the boat is, hence showing the path and the actions chosen to reach the goal.

## Conclusion:

In this experiment, we learned what constraints are, how they are handled, the use of states and finding a solution of a problem using BFS algo.